

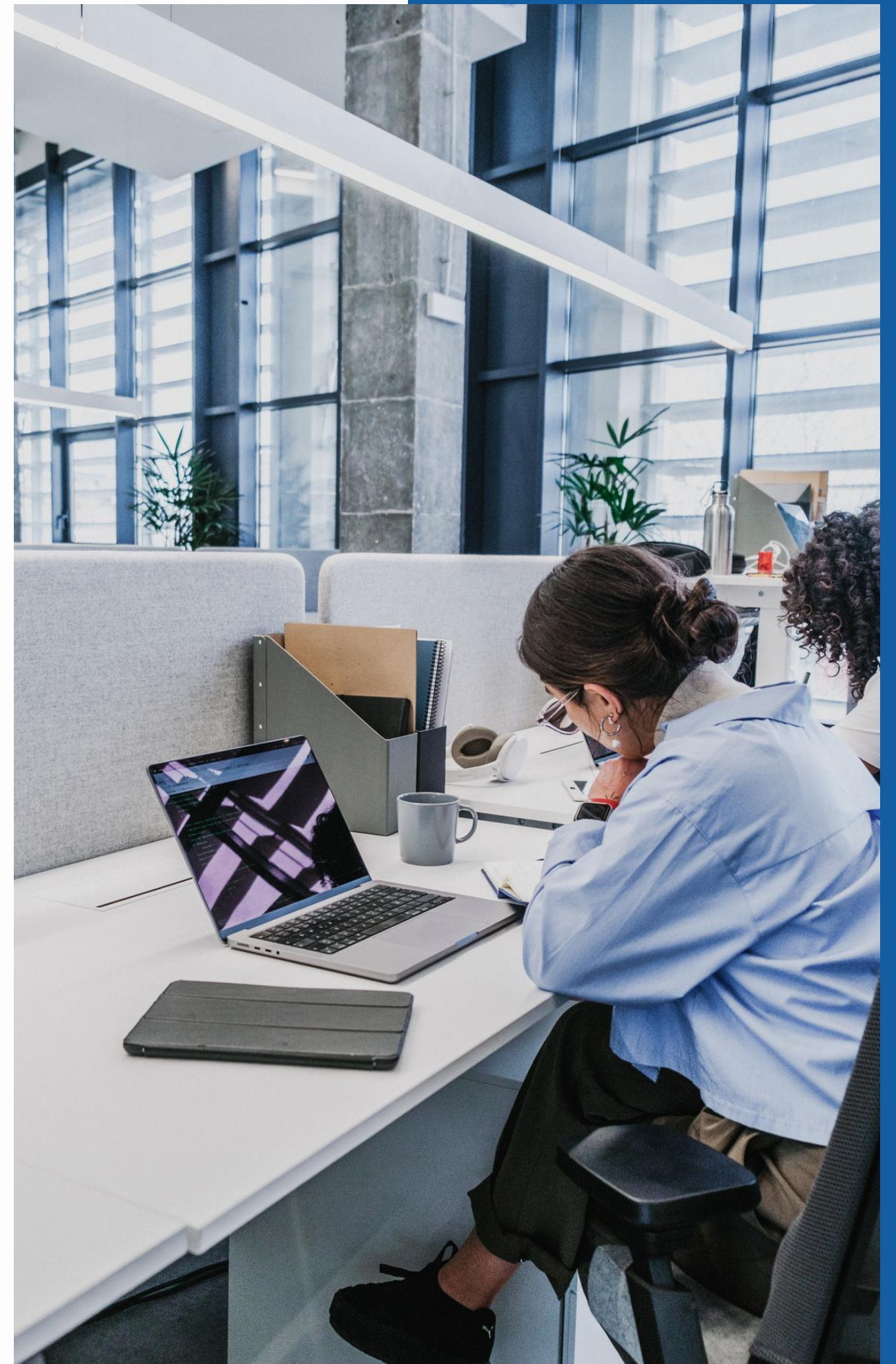
Temporal Analysis of COVID-19 Research Trends Using LDA Topic Modeling

Author:

Devesh Singarh

Overview

- ▶ Introduction
- ▶ Our Aim
- ▶ Data Preprocessing
- ▶ Methodology
- ▶ Visualization of Experimental Results
- ▶ Interpretation of the Results
- ▶ Real World Implications
- ▶ Demo of Our Work



Introduction

Purpose:

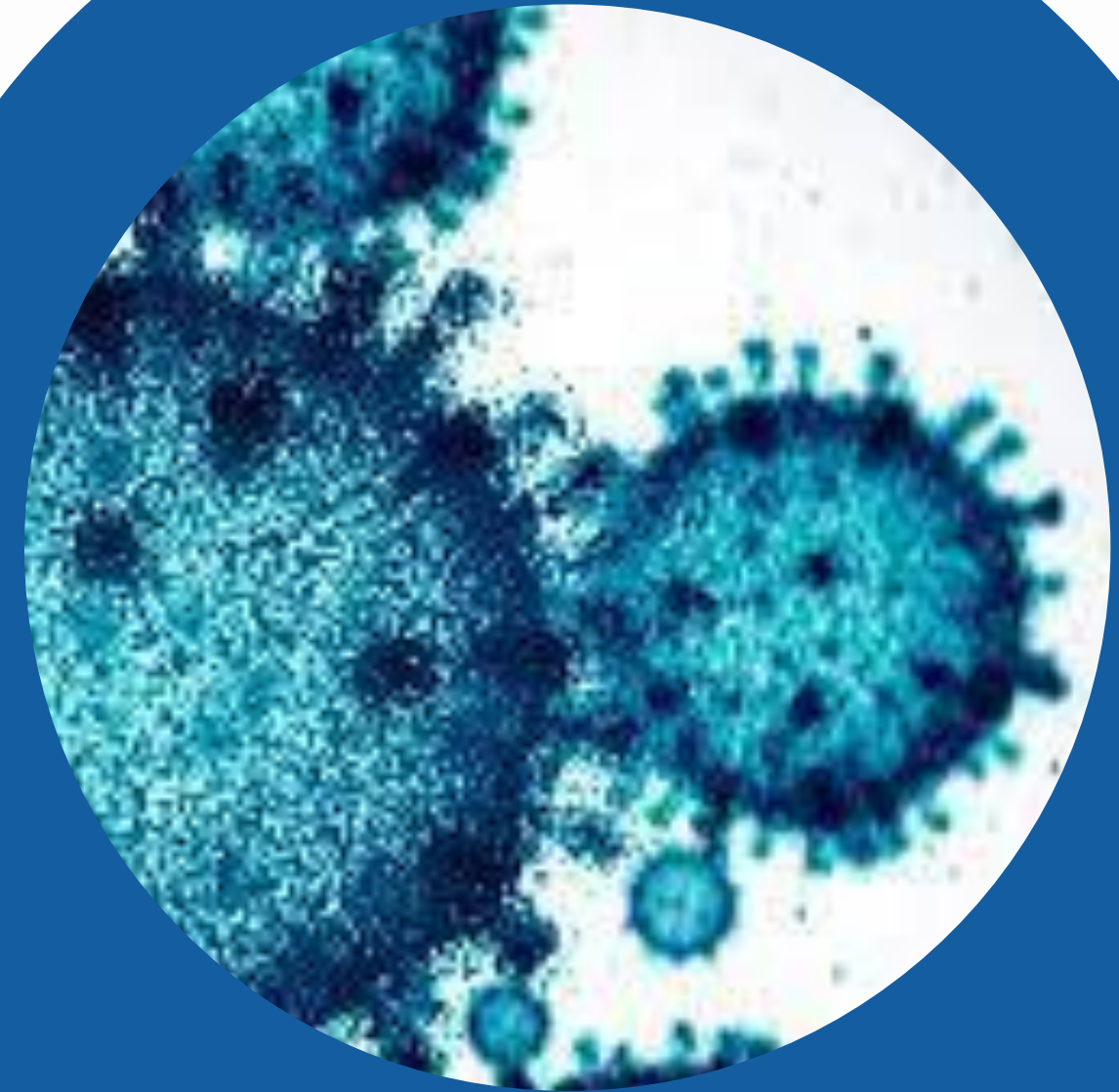
To uncover the dynamic evolution of research themes in COVID-19 scholarly articles.

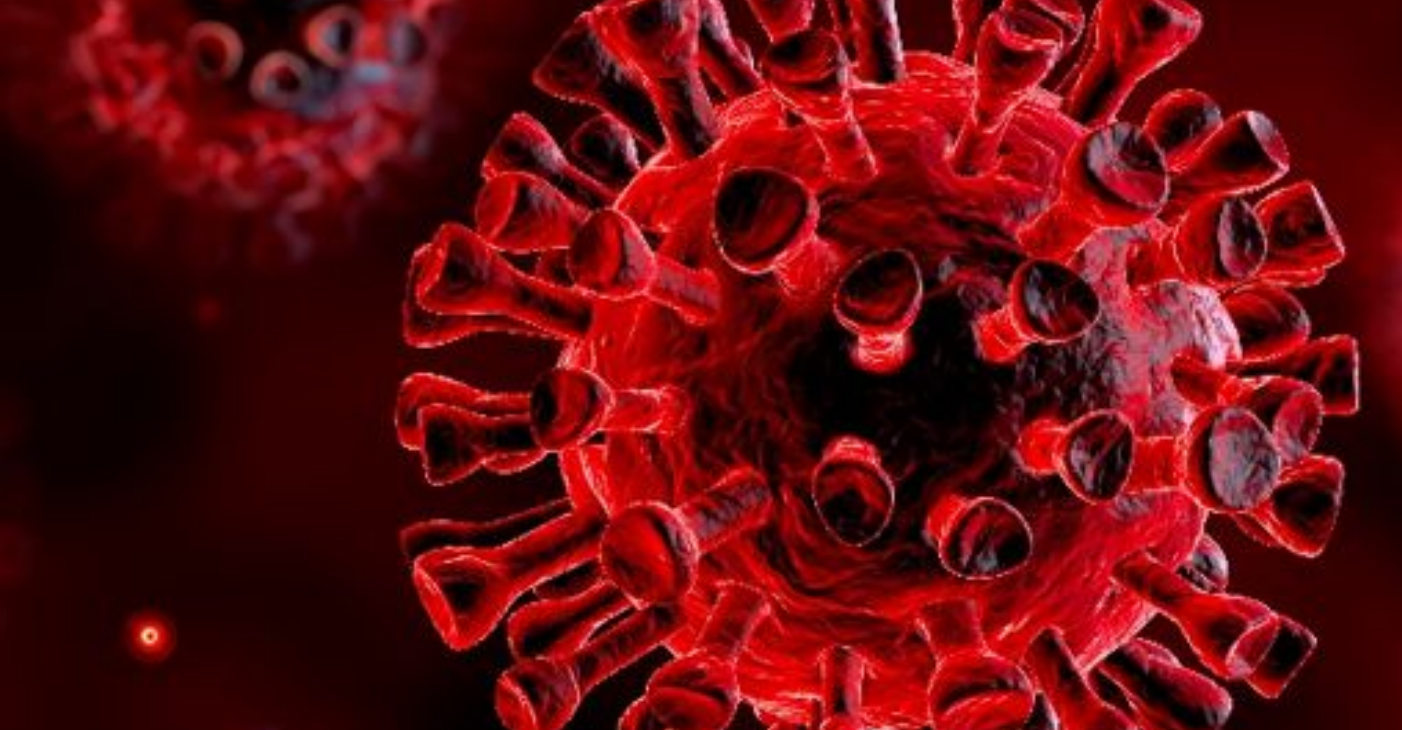
Dataset:

Utilizing the comprehensive CORD-19 dataset, which compiles extensive COVID-19 research from various global sources. We used sok worth of data due to computation constraints.

Approach:

Employing Latent Dirichlet Allocation (LDA) to perform topic modeling, allowing us to extract and analyze key research topics over time.





By solving this problem, we aim to:

- Identify emerging research trends and shifts in focus within the scientific community.
- Understand the impact of global events [e.g., vaccine availability, new COVID-19 variants] on the direction of research.
- Provide valuable insights that can inform future research priorities and help policymakers understand how scientific attention is directed in response to ongoing challenges posed by the pandemic.

Our Aim

Data Cleaning & Selection

```
# Load the CSV file
df = pd.read_csv('metadata.csv', low_memory=False)

# Data Cleaning
# Keep only essential columns
df = df[['title', 'abstract', 'publish_time', 'authors']]

# Drop rows with missing abstracts or publish times
df = df.dropna(subset=['abstract', 'publish_time'])

# Convert 'publish_time' to datetime, filter by years 2020 to 2025, and handle errors
df['publish_time'] = pd.to_datetime(df['publish_time'], errors='coerce')
df = df.dropna(subset=['publish_time'])
df = df[df['publish_time'].dt.year.between(2020, 2025)]

# Create quarterly time periods (Year-Quarter)
df['year_quarter'] = df['publish_time'].dt.to_period('Q')
```

01

Selected relevant fields: *title*, *abstract*, *publish_time*, *authors*

02

Dropped rows with missing *abstract* or *publish_time*

03

Converted *publish_time* to datetime format

04

Filtered dataset to only include years 2020 to 2025

05

Created a new column: *year_quarter* (e.g., 2021Q3)

Text Preprocessing Pipeline

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import RegexpTokenizer
from nltk.stem import WordNetLemmatizer
```

- 01 Combined title and abstract into one text field
- 02 Converted text to lowercase and removed numbers
- 03 Tokenized using regular expressions
- 04 Removed English stopwords (e.g., 'the', 'is', 'and')
- 05 Lemmatized words using WordNet (e.g., "running" → "run")
- 06 Final output: Cleaned text ready for embedding

```
# Download necessary NLTK components
nltk.download('stopwords')
nltk.download('wordnet')

# Initialize NLTK components for text preprocessing
stop_words = set(stopwords.words('english'))
tokenizer = RegexpTokenizer(r'\w+') # Tokenizer to extract words (no punctuation)
lemmatizer = WordNetLemmatizer()

# Define a function for text preprocessing
def preprocess_text(text):
    text = str(text) # Ensure text is treated as a string

    # Remove numbers and tokenize the text
    tokens = tokenizer.tokenize(re.sub(r'\d+', '', text.lower())) # Convert to lowercase and remove numbers

    # Remove stopwords and lemmatize the tokens
    tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in stop_words and len(word) > 2]

    # Join the tokens back into a string
    return ' '.join(tokens)

# Apply the preprocessing function to the title and abstract columns
df_subset['processed_title'] = df_subset['title'].apply(preprocess_text)
df_subset['processed_abstract'] = df_subset['abstract'].apply(preprocess_text)
```

Final Prepared Dataset

```
# Combine preprocessed title and abstract into one text column
df_subset['processed_text'] = df_subset['processed_title'] + " " + df_subset['processed_abstract']

# Display the first few rows of the cleaned and processed dataset
print("First few rows of the cleaned and processed dataset:")
print(df_subset[['title', 'processed_title', 'abstract', 'processed_abstract', 'processed_text']].head())
```

First few rows of the cleaned and processed dataset:

	title \
889956	Real-world deployment of lateral flow SARS-CoV...
716810	Improving Oxygen Capacity at ITBP Referral Hos...
53129	Clinician-Focused Overview and Developments in...
142042	The effectiveness of trauma care systems at di...
942810	Development of ultrahigh-performance liquid ch...

	processed_title \
889956	real world deployment lateral flow sars cov an...
716810	improving oxygen capacity itbp referral hospit...
53129	clinician focused overview development polysom...
142042	effectiveness trauma care system different sta...
942810	development ultrahigh performance liquid chrom...

Methodology



01

Data Segmentation

02

TF-IDF Vectorization

03

Topic Number
Optimization

04

LDA Implementation

05

Topic Dominance

Data Segmentation and TF-IDF

Vectorization



Data Segmentation

Temporal Grouping:

Papers were grouped by publication quarters (Year-Quarter), allowing for a structured analysis over time.



TF-IDF Vectorization

We used the TF-IDF vectorizer to convert the preprocessed text data into a numerical matrix, where important terms are highlighted by assigning higher weights to them, while reducing the weight of commonly occurring words. We set the parameters to ignore terms that appear in more than 95% of the documents and those that appear in fewer than 2 documents. Additionally, both unigrams and bigrams were included to capture more contextual information for topic modeling.

Data Segmentation and TF-IDF Vectorization



Data Segmentation

```
# Convert 'publish_time' to datetime, filter by years 2020 to 2025, and handle errors
df['publish_time'] = pd.to_datetime(df['publish_time'], errors='coerce')
df = df.dropna(subset=['publish_time'])
df = df[df['publish_time'].dt.year.between(2020, 2025)]

# Create quarterly time periods (Year-Quarter)
df['year_quarter'] = df['publish_time'].dt.to_period('Q')
```



TF-IDF Vectorization

```
# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(
    max_df=0.95,      # Ignore terms that appear in >95% of documents
    min_df=2,         # Ignore terms that appear in <2 documents
    max_features=5000,
    ngram_range=(1, 2), # Include unigrams and bigrams
    stop_words='english'
)

# Fit and transform the text data
tfidf_matrix = tfidf_vectorizer.fit_transform(df_subset['processed_text'])
print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
```

TF-IDF matrix shape: (50000, 5000)

Topic Number Optimazation

Coherence Score Evaluation:

- To determine the optimal number of topics, we calculated the coherence score for models with varying numbers of topics. The coherence score measures the degree of semantic similarity between high scoring words in the topic, helping to choose the most interpretable topic model.

Range Tested:

- We tested models with topic numbers ranging from 2 to 10, iterating through each model configuration and recording the coherence scores.

To select the best number of topics for each quarter, which coherence score do we choose?

Lowest Score

Median Score

Highest Score

Submit

● Loading...

Topic Number Optimazation

```
# Dictionary to store LDA models and topic distributions by quarter
optimal_models = {}
time_segment_topic_distributions = {}
optimal_topic_numbers = {}

for period, subset in df_subset.groupby('year_quarter'):
    # Preprocess and prepare the texts for each document
    preprocessed_texts = [preprocess_text(doc) for doc in subset['abstract']]
    texts = [text.split() for text in preprocessed_texts] # Split each preprocessed string into a list of words

    # Create a Gensim Dictionary and Corpus
    dictionary = Dictionary(texts)
    corpus = [dictionary.doc2bow(text) for text in texts]

    # Define a range for the number of topics to evaluate
    topic_range = range(2, 11)
    coherence_values = []
    model_list = []

    # Test different numbers of topics to find the one with the highest coherence score
    for num_topics in topic_range:
        model = LdaModel(corpus=corpus, id2word=dictionary, num_topics=num_topics, passes=10, random_state=42)
        model_list.append(model)
        coherence_model = CoherenceModel(model=model, texts=texts, dictionary=dictionary, coherence='c_v')
        coherence_score = coherence_model.get_coherence()
        coherence_values.append(coherence_score)

    # Select the model that gives the highest coherence score
    max_coherence_index = coherence_values.index(max(coherence_values))
    optimal_model = model_list[max_coherence_index]
    optimal_topic_number = max_coherence_index + 2 # +2 because range starts at 2
    optimal_topic_numbers[period] = optimal_topic_number

    # Store model and topic distributions for the optimal number of topics
    topic_distributions = [optimal_model.get_document_topics(bow) for bow in corpus]
    optimal_models[period] = optimal_model
    time_segment_topic_distributions[period] = topic_distributions

# Print the selected number of topics and the corresponding highest coherence score for this quarter
print(f"Quarter: {period}, Optimal Number of Topics: {optimal_topic_number}, Highest Coherence Score: {coherence_values[max_coherence_index]:.4f}")
```

Quarter: 2020Q1, Optimal Number of Topics: 10, Highest Coherence Score: 0.4182
Quarter: 2020Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5079
Quarter: 2020Q3, Optimal Number of Topics: 10, Highest Coherence Score: 0.5167
Quarter: 2020Q4, Optimal Number of Topics: 9, Highest Coherence Score: 0.5248
Quarter: 2021Q1, Optimal Number of Topics: 5, Highest Coherence Score: 0.5412
Quarter: 2021Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5194
Quarter: 2021Q3, Optimal Number of Topics: 8, Highest Coherence Score: 0.4611
Quarter: 2021Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.5245
Quarter: 2022Q1, Optimal Number of Topics: 8, Highest Coherence Score: 0.5338
Quarter: 2022Q2, Optimal Number of Topics: 10, Highest Coherence Score: 0.4500
Quarter: 2022Q3, Optimal Number of Topics: 6, Highest Coherence Score: 0.5058
Quarter: 2022Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.4855

Best Model Selection

2020Q1 – Optimal Number of Topics: 10

Topic 1: result(5), based(7), model(13), network(6), problem(1), analysis(9), study(8), algorithm(6), system(16), using(7)
Topic 2: study(8), intervention(0), review(2), outcome(1), patient(21), trial(0), group(2), result(5), research(1), clinical(3)
Topic 3: cell(0), protein(0), level(2), model(13), ace(0), system(16), virus(5), expression(1), receptor(0), potential(5)
Topic 4: covid(35), disease(14), cov(1), sars(1), coronavirus(7), infection(2), case(10), virus(5), health(6), respiratory(2)
Topic 5: patient(21), study(8), health(6), year(0), high(3), risk(3), result(5), trauma(0), using(7), breathing(0)
Topic 6: infection(2), virus(5), study(8), human(0), bat(0), result(5), protein(0), dog(0), factor(2), pcr(0)
Topic 7: patient(21), covid(35), day(3), study(8), clinical(3), severe(5), result(5), cov(1), infection(2), sars(1)
Topic 8: case(10), patient(21), clinical(3), covid(35), treatment(1), cov(1), sars(1), drug(0), article(2), result(5)
Topic 9: sars(1), cov(1), group(2), virus(5), quarantine(0), sequence(0), pressure(0), may(3), two(1), study(8)
Topic 10: model(13), case(10), covid(35), epidemic(0), number(8), outbreak(3), data(8), virus(5), china(1), infection(2)

2020Q2 – Optimal Number of Topics: 6

Topic 1: patient(21), covid(35), study(8), clinical(3), result(5), disease(14), outcome(1), treatment(1), case(10), sars(1)
Topic 2: cell(0), expression(1), virus(5), study(8), protein(0), gene(0), showed(2), rna(0), surface(0), effect(4)
Topic 3: cov(1), sars(1), covid(35), disease(14), coronavirus(7), infection(2), virus(5), respiratory(2), severe(5), drug(0)
Topic 4: model(13), based(7), network(6), method(5), data(8), system(16), result(5), algorithm(6), text(0), problem(1)
Topic 5: patient(21), care(6), pandemic(18), covid(35), health(6), healthcare(2), need(6), management(2), practice(5), hospital(3)
Topic 6: covid(35), health(6), case(10), pandemic(18), social(8), disease(14), study(8), data(8), country(11), number(8)

2020Q3 – Optimal Number of Topics: 10

Topic 1: cell(0), protein(0), drug(0), gene(0), human(0), expression(1), study(8), ace(0), receptor(0), effect(4)
Topic 2: le(0), mask(0), de(0), une(0), patient(21), covid(35), été(0), ont(0), dans(0), pour(0)
Topic 3: covid(35), health(6), pandemic(18), care(6), research(1), public(7), social(8), medical(3), disease(14), country(11)
...
Topic 7: vaccine(7), cold(5), covid(35), country(11), storage(4), pandemic(18), data(8), brazil(4), portal(2), disease(14)
Topic 8: covid(35), vaccine(7), cold(5), system(16), work(8), model(13), mobility(15), provide(6), study(8), paper(7)
Topic 9: working(8), smart(8), model(13), practice(5), covid(35), pandemic(18), concept(2), industrial(2), isw(2), activity(7)
Topic 10: wlan(7), model(13), predictive(6), covid(35), network(6), pandemic(18), data(8), algorithm(6), patient(21), disease(14)

LDA Implementation

Data Preparation:

We applied LDA to a document-term matrix derived from the raw counts of terms in each document, which aligns with the assumptions of LDA regarding word distributions.

Model Training:

The LDA model was trained on the optimal number of topics to ensure reliable insights into topic evolution.

LDA Implementation Code

Quarter: 2020Q1, Optimal Number of Topics: 10, Highest Coherence Score: 0.4182
Quarter: 2020Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5079
Quarter: 2020Q3, Optimal Number of Topics: 10, Highest Coherence Score: 0.5167
Quarter: 2020Q4, Optimal Number of Topics: 9, Highest Coherence Score: 0.5248
Quarter: 2021Q1, Optimal Number of Topics: 5, Highest Coherence Score: 0.5412
Quarter: 2021Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5194
Quarter: 2021Q3, Optimal Number of Topics: 8, Highest Coherence Score: 0.4611
Quarter: 2021Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.5245
Quarter: 2022Q1, Optimal Number of Topics: 8, Highest Coherence Score: 0.5338
Quarter: 2022Q2, Optimal Number of Topics: 10, Highest Coherence Score: 0.4500
Quarter: 2022Q3, Optimal Number of Topics: 6, Highest Coherence Score: 0.5058
Quarter: 2022Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.4855

```
# We calculate an average or use the most frequent optimal_topic_number:
average_optimal_topics = int(np.mean(list(optimal_topic_numbers.values())))
# Retrain LDA on the entire dataset with the optimal number of topics using sklearn's LDA for better handling of TF-IDF
lda = LatentDirichletAllocation(
    n_components=average_optimal_topics,
    random_state=42,
    learning_method='online'
)
lda.fit(tfidf_matrix)

# Function to display the top words in each topic
def display_topics(model, feature_names, n_top_words=10):
    for topic_idx, topic in enumerate(model.components_):
        # Get top words without duplicates
        top_words = []
        seen_words = set()

        # Iterate through words in order of importance
        for i in topic.argsort()[::-1]:
            word = feature_names[i]
            if word not in seen_words and len(word) > 2:
                seen_words.add(word)
                top_words.append(word)
            if len(top_words) == n_top_words:
                break

        # Print formatted output
        print(f"Topic {topic_idx + 1}:")
        print("  " + "\n  ".join(top_words))
        print() # Extra space between topics

# Display topics
feature_names = tfidf_vectorizer.get_feature_names_out()
display_topics(lda, feature_names)
```

Python

Topic 1:

- student
- learning
- education
- teaching
- hcws
- cancer
- medical
- online
- virtual
- opioid

Topic 2:

- covid
- health
- pandemic
- data
- model
- social
- research
- country
- study
- public

Topic 3:

- care
- patient
- pandemic
- covid
- surgery
- healthcare
- service
- surgical
- hospital
- nurse

Topic 4:

- patient
- covid
- case
- disease
- infection
- mortality
- risk
- clinical
- day
- severe

Topic 5:

- vaccine
- vaccination
- systematic
- systematic review
- review
- covid vaccine
- meta
- meta analysis
- covid vaccination
- mrna

Topic 6:

- mental
- anxiety
- mental health
- health
- child
- participant
- depression
- study
- stress
- adult

Topic 7:

- learning
- image
- model
- pregnant
- machine
- pregnancy
- deep
- machine learning
- woman
- algorithm

Topic 8:

- cov
- sars
- sars cov
- cell
- protein
- virus
- infection
- viral
- antibody
- drug

Analyzing and Summarizing Topic Dominance

```
# Transform the documents to topic distribution
topic_distributions = lda.transform(tfidf_matrix)
df_subset['dominant_topic'] = np.argmax(topic_distributions, axis=1) + 1

# Count papers per topic
topic_counts = df_subset['dominant_topic'].value_counts().sort_index()
print("Papers per topic:")
for topic_number, count in topic_counts.items():
    print(f"Topic {topic_number}: {count} papers")
```

]

Papers per topic:

Topic 1: 19 papers

Topic 2: 20426 papers

Topic 3: 1832 papers

Topic 4: 11163 papers

Topic 5: 183 papers

Topic 6: 3387 papers

Topic 7: 433 papers

Topic 8: 12557 papers

Visualizing Topic Trends Over Time

```
# Group by quarter and topic
topic_trends = df_subset.groupby(['year_quarter', 'dominant_topic']).size().unstack(fill_value=0)

# Normalize to percentages
topic_percentages = topic_trends.div(topic_trends.sum(axis=1), axis=0) * 100

# Plot the raw count heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(topic_trends, annot=True, cmap="Blues", fmt="g", cbar=True)
plt.title("Topic Distribution Over Time (2020-2025)")
plt.xlabel("Topic")
plt.ylabel("Year-Quarter")
plt.show()

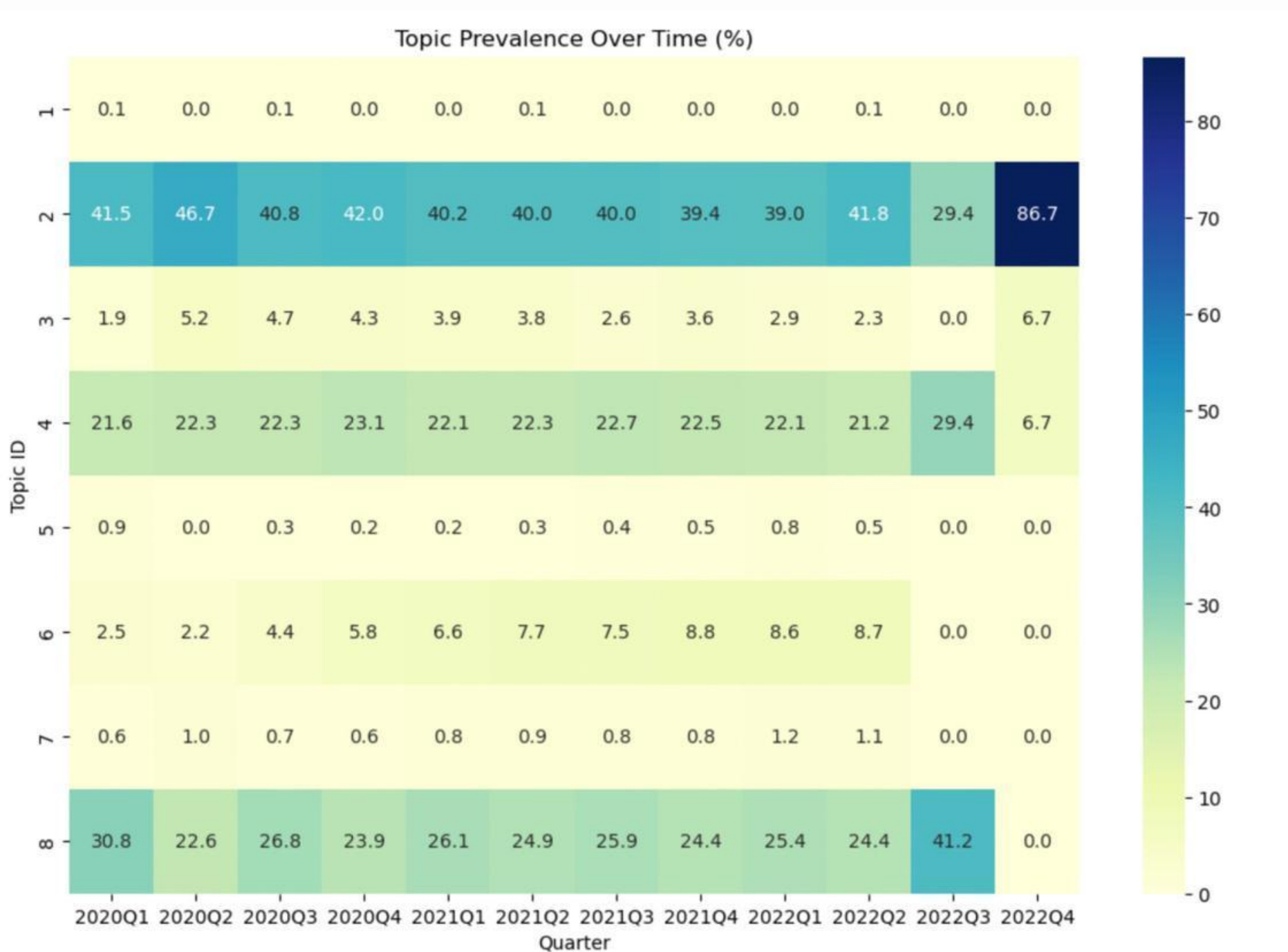
# Plot the percentage heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(topic_percentages.T, cmap="YlGnBu", annot=True, fmt=".1f")
plt.title("Topic Prevalence Over Time (%)")
plt.xlabel("Quarter")
plt.ylabel("Topic ID")
plt.show()

print("Sample of DataFrame with topic assignments:")
print(df_subset[['title', 'year_quarter', 'dominant_topic']].head())
```

Sample of DataFrame with topic assignments:

	title	year_quarter	\
889956	Real-world deployment of lateral flow SARS-CoV...	2021Q4	
716810	Improving Oxygen Capacity at ITBP Referral Hos...	2021Q4	
53129	Clinician-Focused Overview and Developments in...	2020Q4	
142042	The effectiveness of trauma care systems at di...	2021Q3	
942810	Development of ultrahigh-performance liquid ch...	2020Q2	
	dominant_topic		
889956	4		
716810	2		
53129	2		
142042	4		
942810	8		

Heatmaps



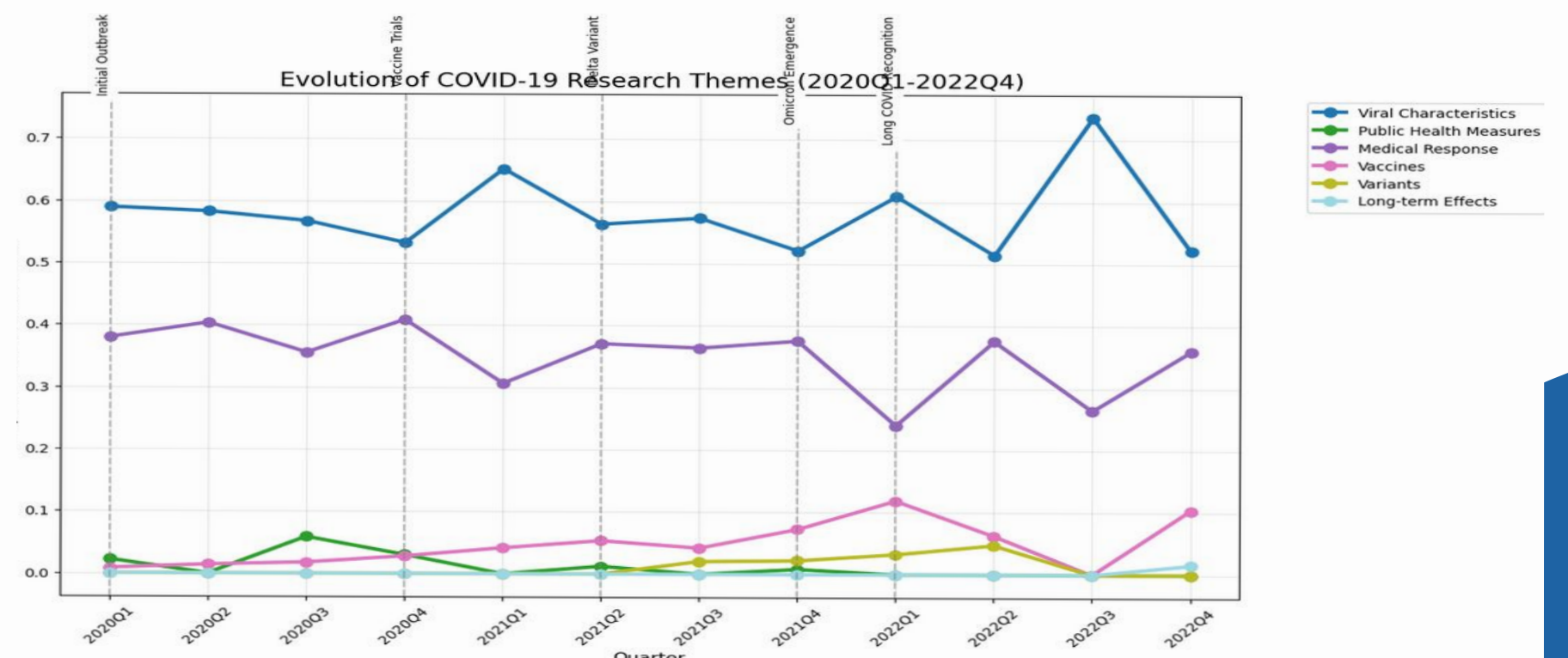
Visualization and Interpretation

Interpretation of Topic Modeling Results

Topic Volatility Reflects Real-World Events:

From the graph “Evolution of COVID-19 Research Themes (2020Q1–2022Q4)”:

- Medical Response and Viral Characteristics consistently dominate research focus (~50–70%) — suggesting foundational topics remained central across all quarters.
- Vaccine-related research rises right before and during 2021Q4, aligning with public vaccination campaigns and the Omicron emergence.



Interpretation of Topic Modeling Results Cont.

Topic Shifts Are Data-Driven, Not Arbitrary:

Our code selects optimal topic numbers per quarter using coherence scores (c_v), maximizing semantic clarity. For example:

- 2020Q2 → 6 topics (coherence = 0.5079)
- 2022Q1 → 8 topics (coherence = 0.5338)

This means topic shifts are based on strong, meaningful clustering, not randomness.

Quarter: 2020Q1, Optimal Number of Topics: 10, Highest Coherence Score: 0.4182

Quarter: 2020Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5079

Quarter: 2020Q3, Optimal Number of Topics: 10, Highest Coherence Score: 0.5167

Quarter: 2020Q4, Optimal Number of Topics: 9, Highest Coherence Score: 0.5248

Quarter: 2021Q1, Optimal Number of Topics: 5, Highest Coherence Score: 0.5412

Quarter: 2021Q2, Optimal Number of Topics: 6, Highest Coherence Score: 0.5194

Quarter: 2021Q3, Optimal Number of Topics: 8, Highest Coherence Score: 0.4611

Quarter: 2021Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.5245

Quarter: 2022Q1, Optimal Number of Topics: 8, Highest Coherence Score: 0.5338

Quarter: 2022Q2, Optimal Number of Topics: 10, Highest Coherence Score: 0.4500

Quarter: 2022Q3, Optimal Number of Topics: 6, Highest Coherence Score: 0.5058

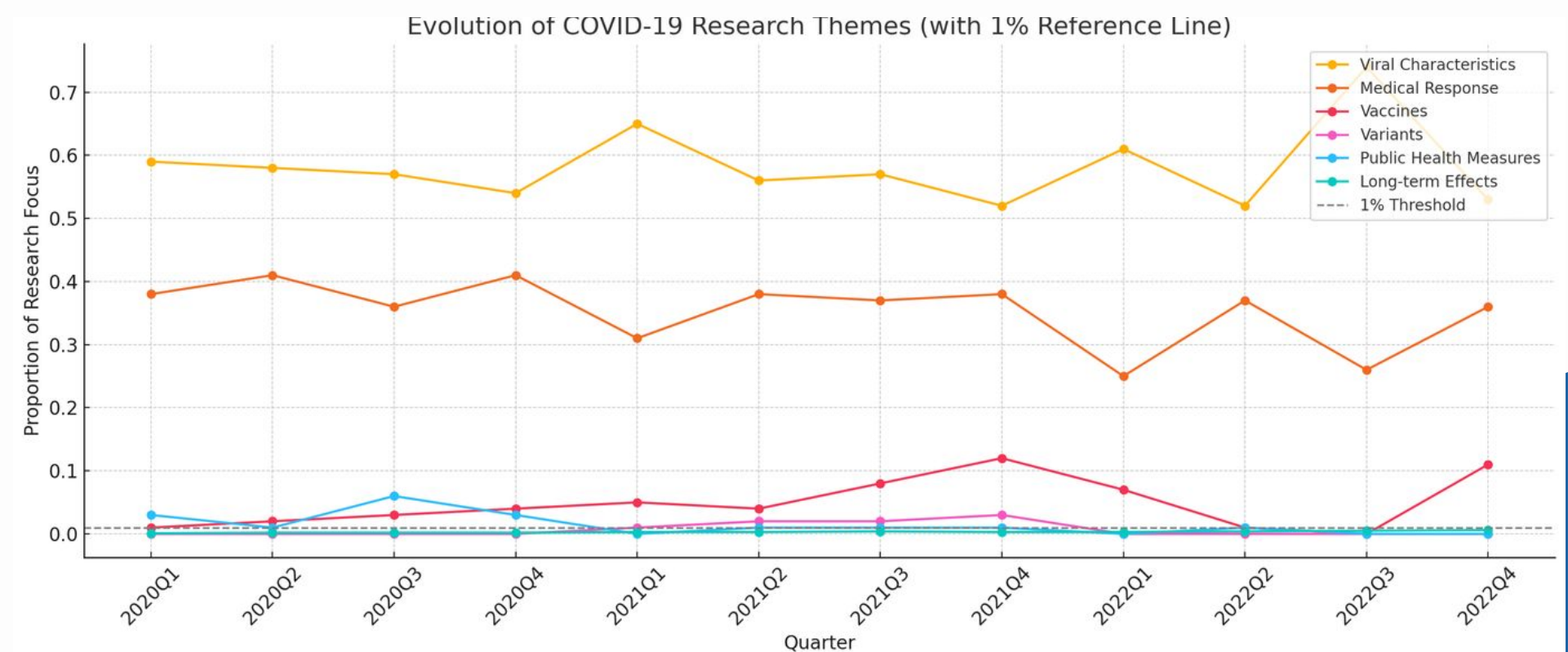
Quarter: 2022Q4, Optimal Number of Topics: 10, Highest Coherence Score: 0.4855

Interpretation of Topic Modeling Results Cont.

Underrepresentation of Long-Term Effects:

Despite growing concern about long COVID, our theme evolution plot shows "Long-term Effects" remained under 1% in all quarters.

- This could signal a gap in academic focus, possibly due to:
 - Lack of early data on long-term outcomes
 - Funding and publication bias toward acute/urgent topics
- Interpretation: There may be a research lag in areas that aren't immediately life-threatening.

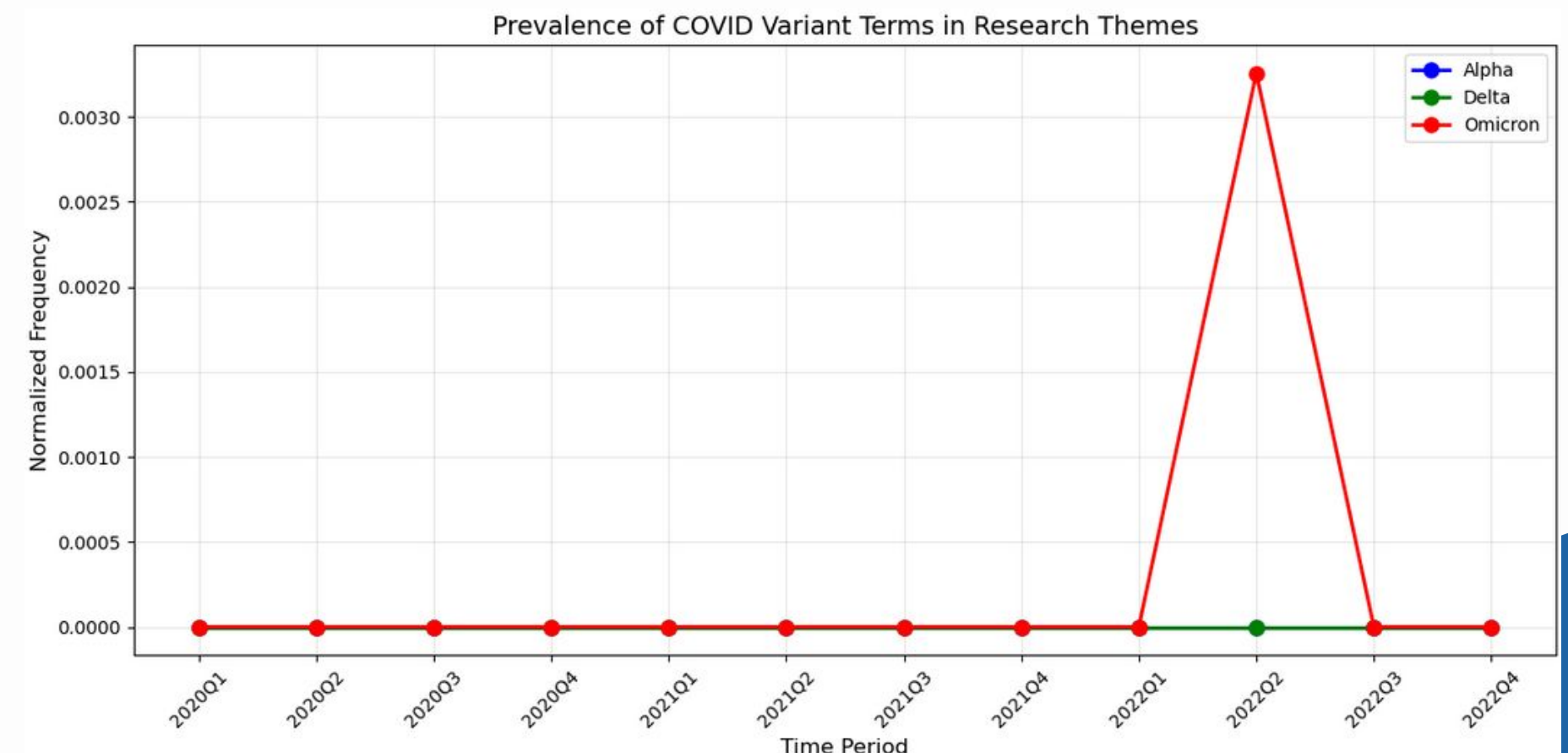


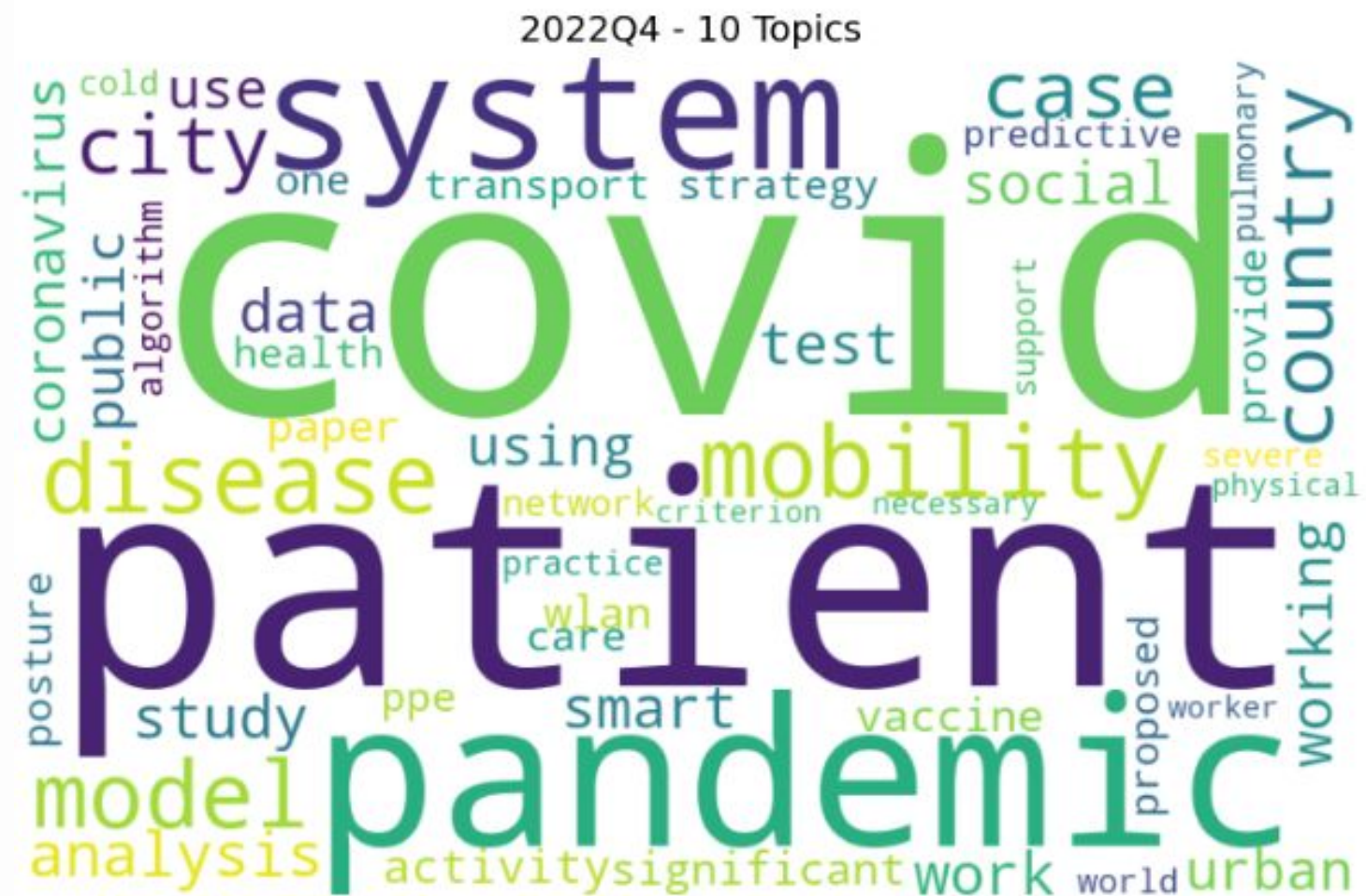
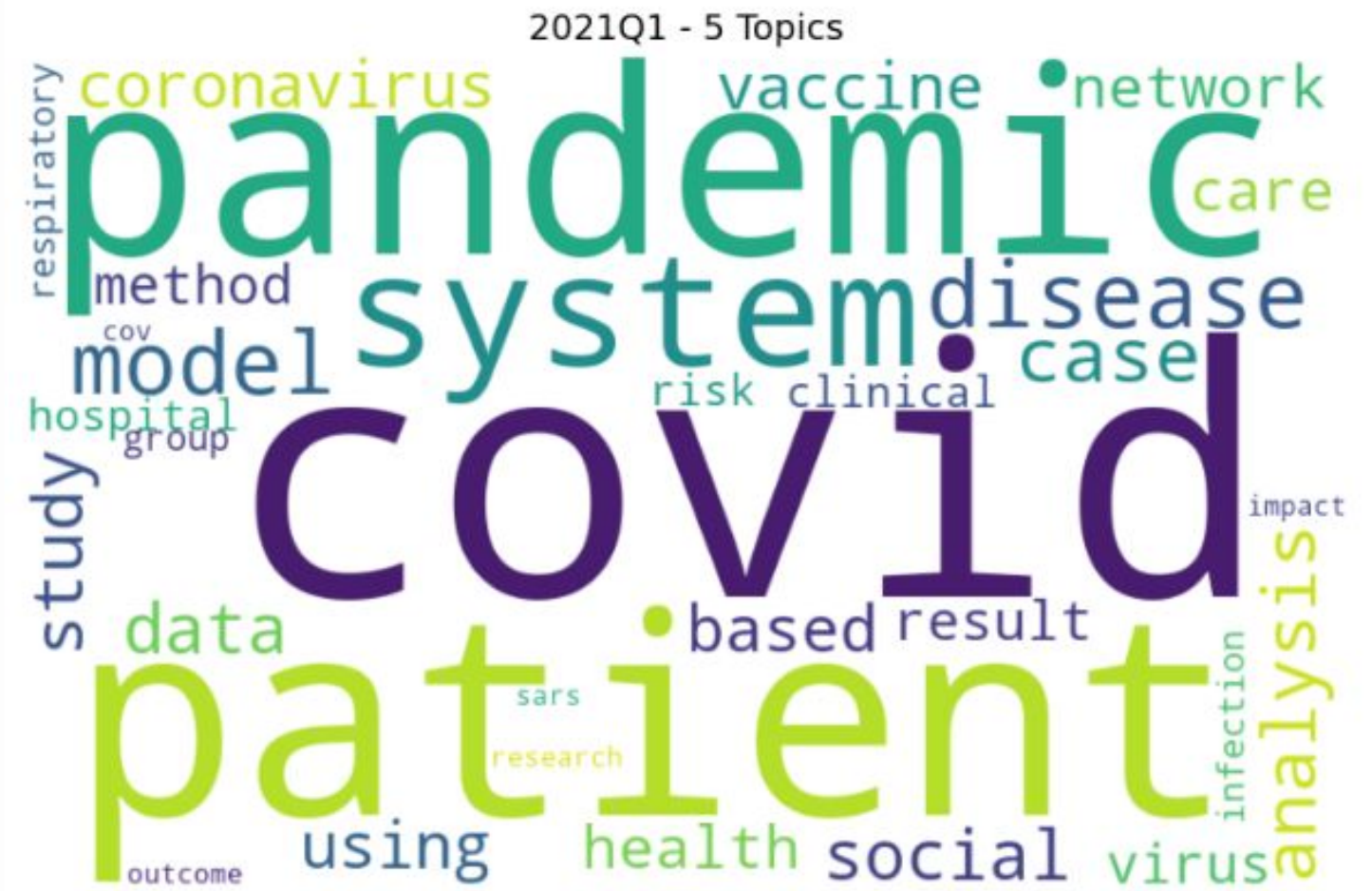
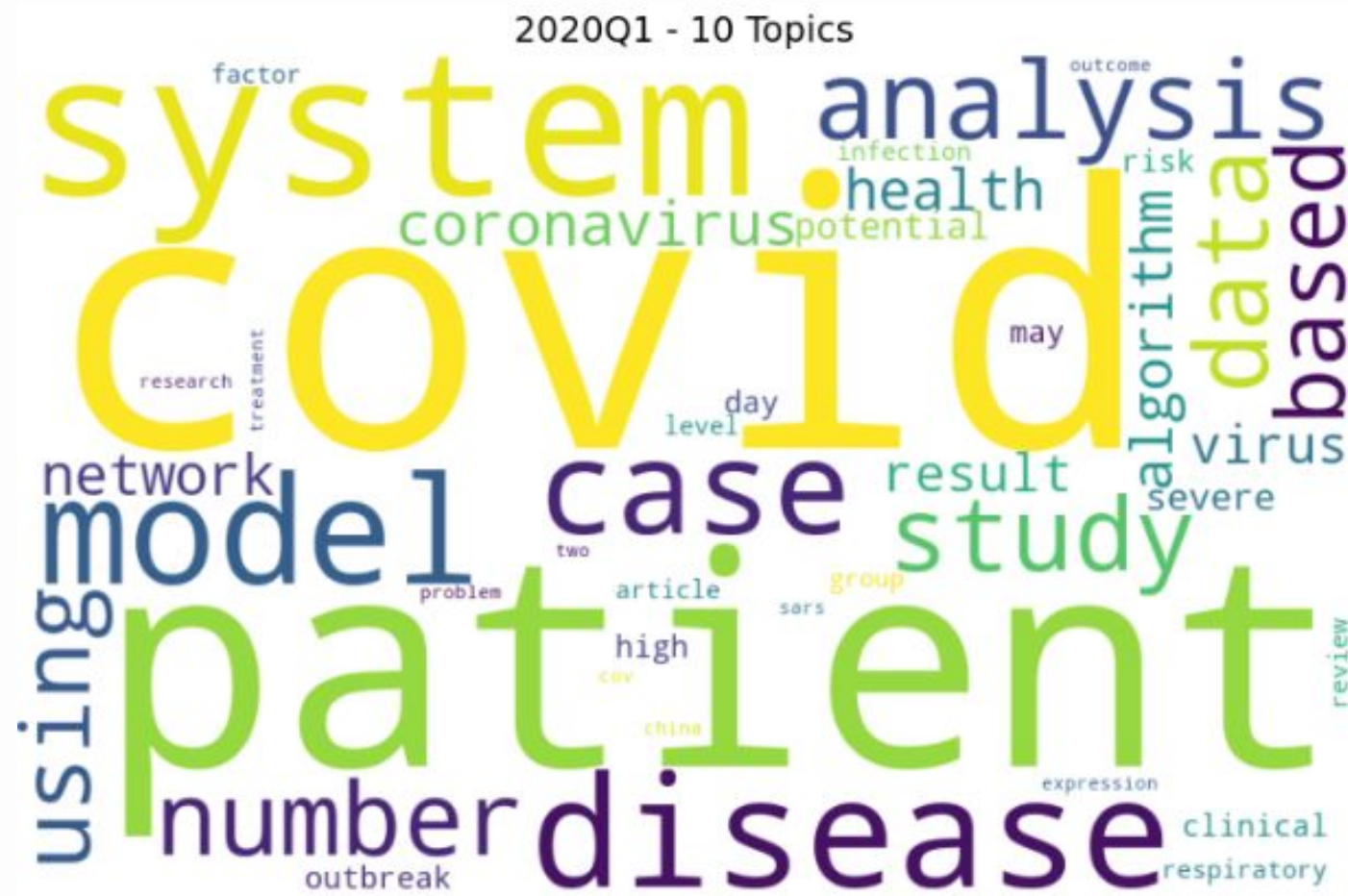
Interpretation of Topic Modeling Results Cont.

Variant-Specific Research Was Short-Lived:

From the “Prevalence of COVID Variant Terms” chart:

- Only Omicron shows a clear research spike - 2022Q2, then quickly disappears.
- Alpha and Delta receive almost no sustained attention in topic keywords.
- This shows that variants created brief surges in research, but didn't lead to long-lasting topic formation in LDA, which captures broader themes





WordClouds

Weigh your Options

When Graphing, You need to choose a method for identifying semantics in words, but you're unsure whether to use BERT (a sophisticated deep learning model) or K-means (a simple clustering algorithm). BERT offers deep contextual understanding but is complex, while K-means is fast and easy to implement but lacks nuance. Which one should you pick?

Option 1: BERT

Captures contextual meaning (semantics), handles polysemy, and is state-of-the-art for NLP.

Though it requires more computational power, needs

fine-tuning for optimal results

Option 2: K-Means

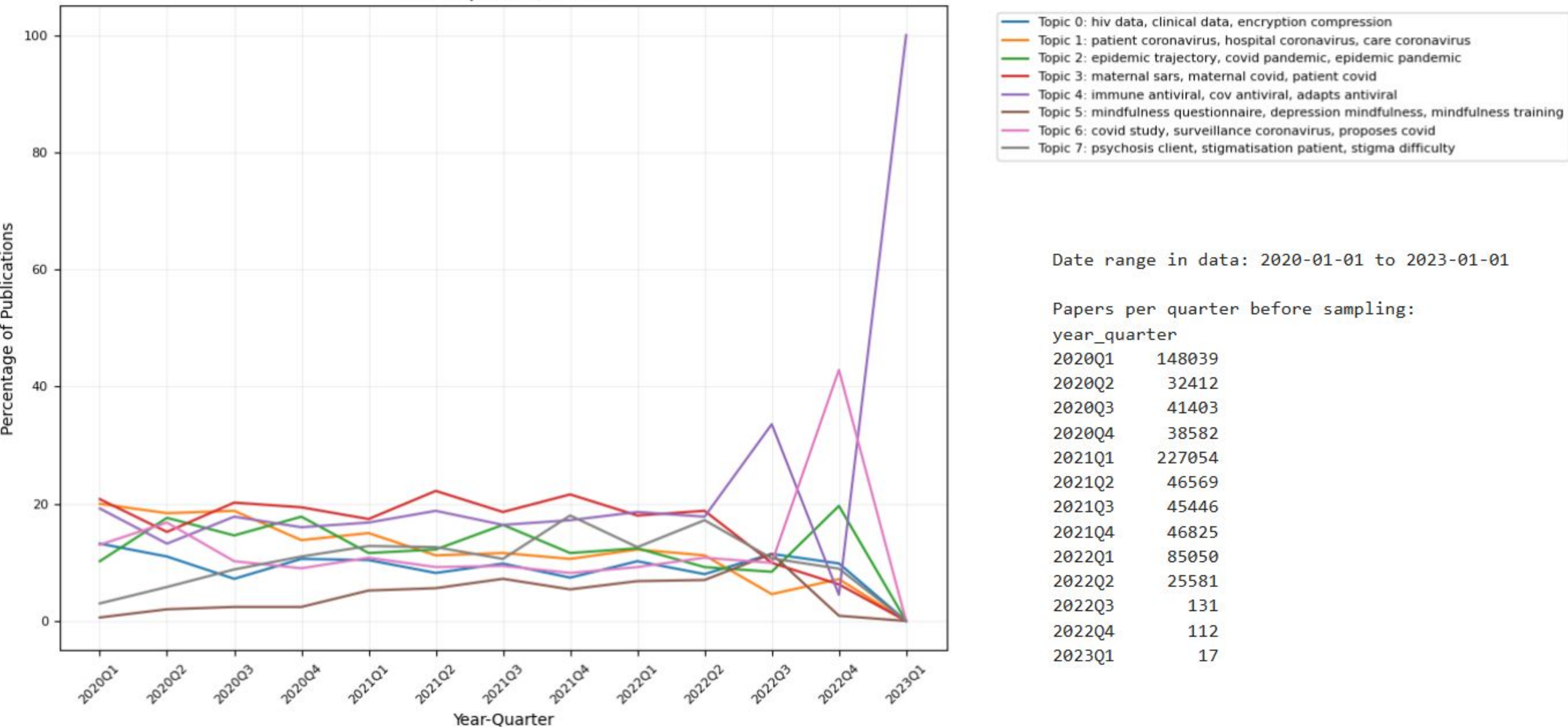
K-means is simple, fast, works well for basic word grouping. However, it ignores context, struggles with multiple meanings of

words.



Research Theme Evolution (2020-2025)

500 Papers/Quarter



What do the Graphs Say?

Some Spike?:

Some topics experienced sudden spikes. (e.g., immune antiviral research and COVID surveillance studies in 2022Q3). This can indicate moments when certain research areas became urgent ie a new variant or new outbreak in a new area.

Others Decline:

Other topics declined over time, suggesting that the research communities moved away from those issues as solutions were found or as new challenges emerged and those topics became buried.

Key Takeaways

What's in it for Humanity?



Public Health

This chart provides insights into the evolution of research priorities over time, allowing public health agencies to identify emerging and declining areas of focus. For example, spikes in research on mental health or COVID-19 surveillance might indicate growing concerns that require policy responses. By tracking shifts in infectious disease research, policymakers can anticipate future healthcare needs and allocate funding accordingly.



The Research Community

For researchers, visualizations help in understanding how scientific interest evolves over time and identifying which topics gain or lose momentum. This helps in making data-driven decisions. (work aligns with current issues) The chart also can highlight potential interdisciplinary opportunities. Lastly, funding agencies/journal editors can use these insights to prioritize support for emerging or underrepresented fields

Reflection: Where to From Here?



Semantics and Subtopics

Exploring into topics that improve Semantic readings with LLMs such as GPT-4. To go even further with this research, we can fine-tune (train) the LLMs on the dataset for even better language comprehension. Nested LDAs also help with nested subtopics, distinguishing rising/falling subfields. LSTMs or transformer-based models can predict future research trajectories, thus merging the gap between analysis and projection.



Could I do BERTer....

SciBERT is not the only kind of BERT that can be used. Sentence transformers like SBERT can improve clustering quality through semantic similarity. For dynamic topic evolution, Dynamic Topic Modeling (DTM) or BERTopic outperforms static clustering (ie. 500) by capturing smooth transitions. Time-aware embeddings (e.g., Temporal BERT) also help detect subtle shifts in research focus.

Self-Reflection: Can I do Better?



Better Sampling Techniques

Alternative sampling and data-collecting techniques can enhance analysis by replacing the fixed 500-paper sampling with dynamic/stratified sampling. This approach prioritizes emerging topics while suppressing overrepresented ones.

Lastly there is active learning, where a model dynamically selects the most informative papers per quarter to optimize dataset relevance.

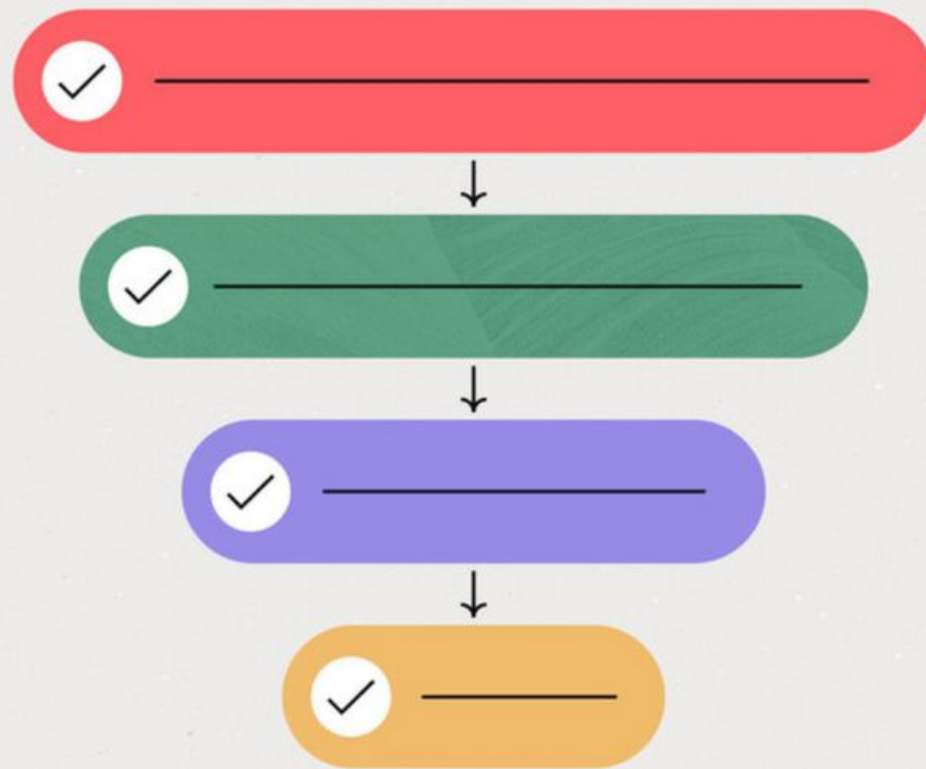


Sophisticated Algorithms

The MiniBatchKMeans can be replaced with more refined approach like BERTopic for density-aware clustering. Together, they merge traits from UMAP as well as TF-IDF, this provides better topic coherence. Otherwise, an embedding-based approach like EmbedTopic cluster directly in the embedding space. This cancels the need for predefined cluster counts and also captures topic shifts better over time. Overall, these techniques can offer the visuals and results a more flexible and accurate analysis of topic evolutions.

Potential Real-World Implications

Informing Research Priorities



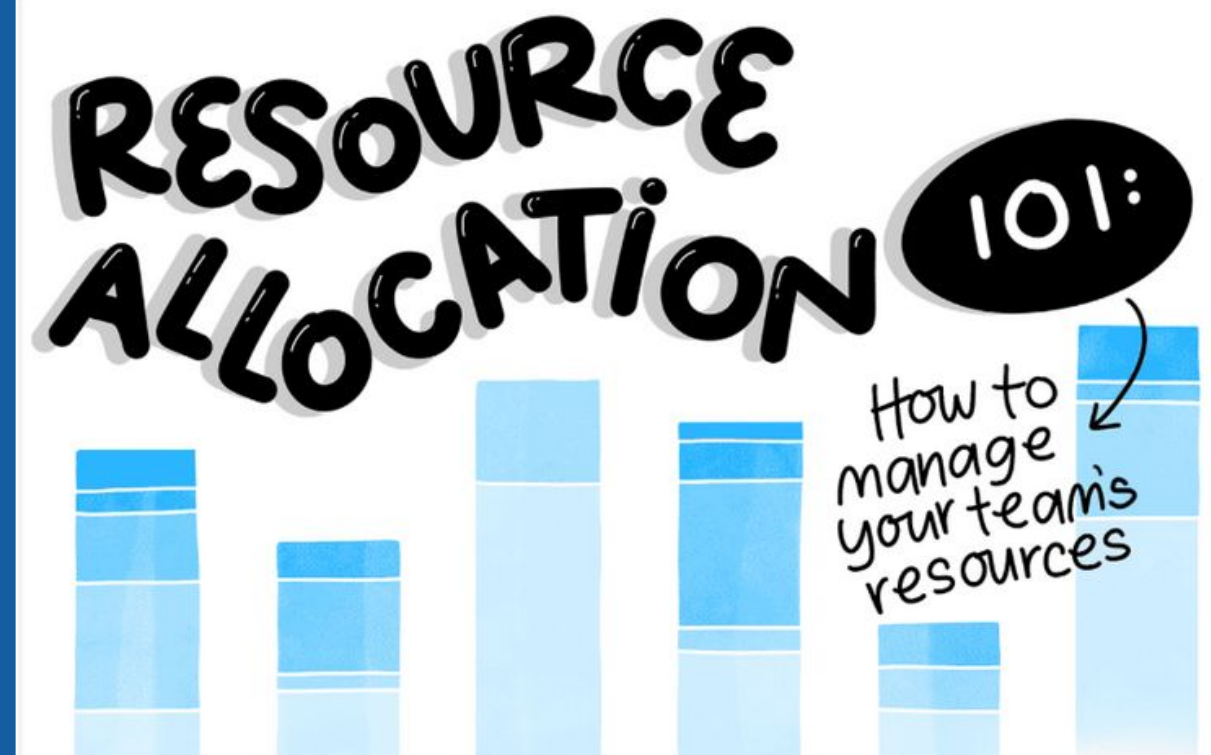
Identifying how research themes evolve over time enables research institutions, academic bodies, and funding agencies to understand which areas have been extensively covered and which need further exploration.

Adapting to Emerging Trends



Our project's ability to track the evolution of COVID-19 research themes over time provides crucial insights into how the scientific community adapts to emerging trends and new information about the virus. This adaptability is essential for timely responses to new challenges such as variants, treatment responses, and public health strategies.

Resource Allocation



The ability to understand the shifts in research focus over time allows for better strategic planning and allocation of resources, whether financial, human, or material, particularly in research and healthcare settings.

References:

Dataset: CORD-19 Dataset

<https://www.kaggle.com/datasets/allen-institute-for-ai/CORD-19-research-challenge/data?select=metadata.csv>